Assignment 1 Answers and Explanation

Task 1

setupAccounts.sh

Explanation:

The purpose of `setupAccounts.sh` is to create the `students` and `tutors` groups, read account details from `Accounts.txt`, create user accounts with the appropriate settings, assign initial passwords, and log group and user creation events to the system log.

Commands Used and Explanations:

getent group students

getent group tutors

The `getent group` command checks whether the specified group exists on the system. The script redirects the output to `/dev/null` and uses `!` to determine whether the group is missing.

groupadd students

groupadd tutors

The `groupadd` command creates the `students` and `tutors` groups if they do not already exist. The script checks whether each command succeeds before continuing.

logger "Created group: students"

logger "Created group: tutors"

The `logger` command writes messages to the system logging service, recording when a group is created.

while IFS=, read -r username account_type || [[-n "$username"]]; do

The `while` loop reads `Accounts.txt` one line at a time. `IFS=,` uses the comma as the field separator, allowing the username and account type to be stored in separate variables. The condition after `||` allows the script to process a final line even if it does not end with a newline character.

[[-z "$username"]] && continue

This skips empty lines so that the script does not attempt to create an account without a username.

```
case "$account_type" in
    student)
        group="students"
        ;;
    tutor)
        group="tutors"
        ;;
    *)
        echo "Invalid account type for $username"
        continue
        ;;
esac
```

The `case` statement checks the account type. Students are assigned to the `students` group, while tutors are assigned to the `tutors` group. Invalid account types are reported and skipped.

```
id "$username"
```

The `id` command checks whether the specified user exists. The script redirects its output and error messages to `/dev/null` and creates the account only if the user does not already exist.

```
useradd -m -s /bin/bash -g "$group" "$username"
```

The `useradd` command creates a new user account. The options mean:

- `-m`: Create the user's home directory.

- `-s /bin/bash`: Set Bash as the user's login shell.

- `-g "$group"`: Assign the specified group as the user's primary group.

- `"$username"`: Specify the username to create.

```
echo "$username:Welcome123" | chpasswd
```

The `echo` command supplies the username and initial password in the format expected by `chpasswd`. The `chpasswd` command sets the user's password to `Welcome123`.

```
logger "Created user: $username in group $group"
```

This records the creation of each user and their assigned primary group in the system log.

```
echo "Failed to create group: students"
```

```
echo "Failed to create user: $username"
```

The script displays error messages when group or user creation fails. It exits if group creation fails, skips invalid account types, and reports password-setting failures.

```
chmod +x setupAccounts.sh
```

```
bash -n setupAccounts.sh
```

```
sudo ./setupAccounts.sh
```

- `chmod +x` makes the script executable.

- `bash -n` checks the script for Bash syntax errors without executing it.

- `sudo ./setupAccounts.sh` runs the script with the privileges required to create groups, create users, and set passwords.

```
id alice
```

Check the user's home directory and login shell:

```
getent passwd alice
```

Check system logs for creation messages:

```
sudo grep -E 'Created group:|Created user:' /var/log/syslog
```

The log verification command assumes the system stores these messages in `/var/log/syslog`; the location may differ depending on the system's logging configuration.

crontab entry:

```
0 7,19 * * 1-5 /bin/bash /home/csci322/reportTutors.sh
```

Explanation:

This crontab entry schedules reportTutors.sh to execute at 7:00 AM and 7:00 PM every weekday, from Monday to Friday. The first field (0) specifies the minute, while 7, 19 specifies the hours in 24-hour format. The next two asterisks (* *) indicate every day of the month and every month, respectively. The field 1-5 specifies Monday through Friday. Finally, /bin/bash /home/csci322/reportTutors.sh runs the script using the Bash shell.

reportTutors.sh

Explanation:

```
while IFS=, read -r username account_type || [[ -n "$username" ]]; do
```

The while loop reads Accounts.txt one line at a time. IFS=, sets the comma as the field separator, allowing the username and account type to be stored in separate variables. The additional condition handles a final line that does not end with a newline character.

```
if [[ "$account_type" == "tutor" ]]; then
```

The if statement checks whether the account type is tutor. Only users with this account type are selected.

```
echo "$username"
```

The echo command prints the username of each matching tutor, with each username on a separate line.

```
done < Accounts.txt > /tmp/tutors.txt
```

< Accounts.txt redirects the contents of Accounts.txt into the loop.

> /tmp/tutors.txt redirects the output into /tmp/tutors.txt, creating the file if it does not exist or overwriting it if it already exists.

```
chmod +x reportTutors.sh
```

```
bash -n reportTutors.sh
```

```
./reportTutors.sh
```

chmod +x makes the script executable.

bash -n checks for Bash syntax errors without executing the script.

./reportTutors.sh runs the script to generate /tmp/tutors.txt.

Task 2

Qn: Determine how many DELETE requests appear in the log. Show the command used.

```
grep -c '"DELETE ' university_access.log
```

Explanation: The grep -c command counts the no. of lines in university_access.log containing the pattern "DELETE. This identifies HTTP DELETE requests, and the log contains 4 DELETE requests.

Qn: Determine how many DELETE requests appear in the log. Show the command used.

```
awk '$6 == "\"GET" && $7 == "/downloads/course-guide.pdf" {host=$1; timestamp=$4" "$5} END {if (host != "") print host, timestamp}' university_access.log
```

Explanation: The grep -c command counts the number of lines in university_access.log that contain the pattern "DELETE. This identifies HTTP DELETE requests. The output 4 indicates that there are four DELETE requests in the log.

Qn: Write a one-line command to find the most recent GET request for /downloads/courseguide.pdf. Display the hostname and timestamp.

```
awk '$6 == "\"GET" && $7 ~ /^\/courses\/[^/]+$/ {sub(/^\/courses\//, "", $7); print $7}' university_access.log | sort -u > courses.txt
```

Explanation: The awk command checks each log entry for a GET request to /downloads/courseguide.pdf. It stores the hostname in field $1 and the timestamp in fields $4 and $5. As the log is processed, each matching request replaces the previous match, so the command displays the hostname and timestamp of the last matching request in the file. This identifies the most recent request if the log entries are in chronological order.